Assignment 1: Self-Supervised Learning with SimSiam on STL-10

B125808026 TAI, WEI-HSUAN

Abstract

We re-implement SimSiam [2] with a ResNet-18 backbone and pretrain it on the 100k unlabeled images of STL-10. With only 1,000 labeled images, a linear probe on the frozen SimSiam encoder reaches $80.98 \pm 0.35\%$ *test accuracy (200 epochs, 3 pretraining seeds), while the same network trained from scratch reaches only* $55.66 \pm 0.93\%$ *even with a tuned batch size and lr. We also reproduce the complete collapse reported in the original paper: removing either the stop-gradient or the predictor drops the linear probe to exactly chance level (10.00%). As a new ablation, we remove one family of augmentations at a time. Without cropping, SimSiam suffers* dimensional collapse*: the backbone's effective rank falls from 318 to 6.7, and accuracy* drops *with longer training. Replacing the crop shows that the views must* differ*: one crop shared by both views is as bad as none, independent random masking recovers two thirds of the gap, and permuted image tiles, which a CNN with global pooling easily ignores, barely help.*

1. Introduction

SimSiam [2] is one of the simplest self-supervised methods. It trains a Siamese network to make the representations of two augmented views of an image agree, and uses no negative pairs, no large batches and no momentum encoder. Contrastive methods such as SimCLR [1] rely on negatives to avoid the constant solution, BYOL [5] on a momentum encoder; SimSiam claims that a stop-gradient and a predictor alone suffice.

Without negatives, the difference between the two augmented views is SimSiam's *only* learning signal. SimCLR found crop plus color the most important augmentation pair for contrastive learning, and InfoMin [12] argues that good views should share task-relevant information and differ in everything else. The SimSiam paper, however, ablates the architecture and the loss but not the augmentations that create this signal. In this report we re-implement SimSiam on STL-10 [3] and study both sides. We (i) show that SimSiam pretraining on 100k unlabeled images beats a tuned from-scratch baseline by 25.3 points with 1,000 labels (Sec. 5.2); (ii) reproduce the complete collapse caused by removing the stop-gradient or the predictor (Sec. 5.3); (iii) show that without cropping SimSiam suffers *dimensional collapse* that worsens with training and that neither the loss nor the output std detects (Sec. 5.4); and (iv) decompose the crop and find that what matters is a content difference between the two views, not data diversity (Sec. 5.5).

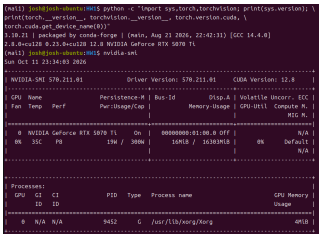


Figure 1. Experimental environment (`nvidia-smi`).

2. Environment and Codebase

All experiments ran on one NVIDIA GeForce RTX 5070 Ti (16 GB, driver 570.211.01, CUDA 12.8) with Python 3.10.21, PyTorch 2.8.0 and torchvision 0.23.0 (Fig. 1). The code extends the course starter code. `model.py` holds SimSiam and its loss, with `stop_grad` and `predictor` flags for the ablations; `data.py` the STL-10 loaders, validation split and two-view augmentation with an `aug` option (Sec. 3); `pretrain.py` logs loss, kNN accuracy and output std every epoch; `linear_probe.py` trains a linear (or BN) probe on the frozen backbone and `sft.py` the same ResNet-18 from scratch. `analyze_rank.py` (erank), `eval_probes.py` (AUROC/F1), `stats_tests.py` and `analyze_crop_decomp.py` (tests) do the analysis, and `run_*.sh` drive all experiments.

3. Method

3.1. SimSiam

Given an image x, two random augmentations produce views x_1 and x_2. An encoder f, consisting of a backbone and a projection MLP, maps each view to $z_i = f(x_i)$, and a prediction MLP h maps it to $p_i = h(z_i)$. With the negative cosine similarity $\mathcal{D}(p, z) = -\frac{p}{\|p\|_2} \cdot \frac{z}{\|z\|_2}$, the symmetric

loss is

$$\mathcal{L} = \tfrac{1}{2}\mathcal{D}(p_1, \mathrm{sg}(z_2)) + \tfrac{1}{2}\mathcal{D}(p_2, \mathrm{sg}(z_1)), \quad (1)$$

where $\mathrm{sg}(\cdot)$ is the stop-gradient. The minimum of $\mathcal{L}$ is -1.

We use a ResNet-18 backbone [6] with its fc layer removed (512-d output), a 3-layer projection MLP ($512 \to 2048 \to 2048 \to 2048$, BN on every layer) and a bottleneck prediction MLP ($2048 \to 512 \to 2048$).

3.2. Ablations

Architecture. Following Sec. 4.1–4.2 of the original paper, we (a) remove the stop-gradient so that gradients flow through both branches, and (b) replace the predictor h with the identity.

Augmentation. The original paper's EM-like hypothesis (Sec. 5) views SimSiam as approximately solving

$$\min_{\theta} \ \mathbb{E}_{x,\mathcal{T}} \left\| f_\theta(\mathcal{T}(x)) - \eta_x \right\|^2, \quad (2)$$

where f_θ is the encoder and η_x is the expected representation of x over augmentations $\mathcal{T}$. If the augmentations are too weak, this expectation becomes trivial. To test this, we remove one family of augmentations at a time: **no_color** drops color jitter and grayscale, and **no_crop** drops RandomResizedCrop, so that both views show the full image and differ only in flip and color.

Decomposing the crop. A crop shows only *part* of the content, *rescales and shifts* it, makes the two views differ and adds diversity across epochs. To separate these factors we replace it (keeping flip and color) with: **crop_in_place** keeps a RandomResizedCrop-style region at its original position and scale and fills the rest with the dataset mean (partial content only); **zoom_out** shrinks the whole image to area fraction a (crop-like aspect ratio) and pastes it at a random position on a mean-filled canvas (rescaling and shifting only); **mask** keeps a random fraction a of the 36 16×16 patches and fills the rest with the mean (partial content, scattered, no rescaling); **same_crop** applies one random crop to *both* views, so the data diversity matches the baseline but the views no longer differ in crop; **shuffle** cuts each view into 2×2 tiles and permutes them independently (same content, different layout), and **shuffle_shared** uses one permutation for both views (same seams, no layout difference) as its control. In crop_in_place, zoom_out and mask, $a \sim U(0.2, 1)$ as in the crop, drawn independently for each view.

3.3. Measuring Collapse

Complete collapse maps every input to one vector; in *dimensional collapse* [9, 10] the embeddings stay spread out but occupy a low-dimensional subspace. The std of the ℓ_2-normalized output ($1/\sqrt{d}$ when spread over the sphere, 0 under complete collapse) detects only the former, so we also measure the effective rank (erank) [11], which RankMe [4] showed predicts downstream performance without labels. With σ_k the singular values of the centered, ℓ_2-normalized features of the 8,000 test images,

$$\mathrm{erank} = \exp\Big(-\sum_k p_k \log p_k\Big), \quad p_k = \frac{\sigma_k}{\sum_j \sigma_j}. \quad (3)$$

We focus on the 512-d backbone feature, the probe's input.

4. Experimental Setup

Dataset. We use STL-10 [3] at 96×96. Pretraining uses the 100k unlabeled images. Classifiers train on the official fold 0 (1,000 images). Validation uses 500 held-out, label-balanced training images, and testing uses all 8,000 test images.

Pretraining. Following the original paper's CIFAR setting, we use SGD (momentum 0.9, weight decay 5×10^{-4}), batch size 512, lr $= 0.03 \times 512/256$, a 10-epoch warmup and cosine decay. The augmentation is RandomResizedCrop (scale 0.2–1), horizontal flip, color jitter $(0.4, 0.4, 0.4, 0.1)$ with p=0.8, and grayscale with p=0.2, without the Gaussian blur of its ImageNet setting. As in the paper, the predictor's lr is not decayed. A kNN monitor [14] (k=200, τ=0.1) uses fold 0 as the memory bank and evaluates on the validation images after every epoch.

Evaluation. The *linear probe* trains a linear layer on the frozen backbone with SGD (lr 30 as in MoCo [7], no weight decay) for 50 epochs. The *BN probe* adds a parameter-free BN before the linear layer as in MAE [8], with lr chosen on validation. The from-scratch baseline (*SFT*) trains the whole ResNet-18 with the pretraining optimizer settings (batch 512, about 2 steps per epoch) for 50 epochs. Both train with RandomResizedCrop (scale 0.08–1) and horizontal flip. The pretrained encoder is always the final-epoch checkpoint; for the classifier we test the *probe/SFT epoch* with the best validation accuracy.

Statistics. For the 200-epoch comparisons we pretrain with seeds 123/456/789, and linear-probe each model with 5 fine-tuning seeds (12/34/56/78/90). The *pretraining seed* is the unit of replication: we report the mean $\pm$ std over the 3 per-model means and compare settings with two-sided Welch t-tests. We also compute macro one-vs-rest AUROC and macro F1 for every probe, and check every comparison under all three metrics with Holm correction.

5. Experiments

The original paper uses ImageNet and ResNet-50, so we compare only qualitative trends.

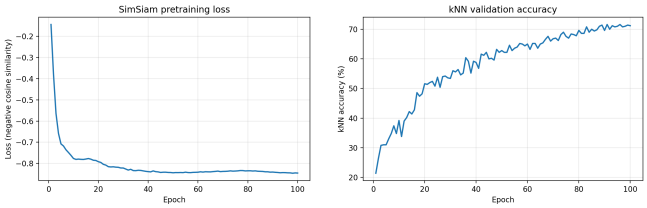


Figure 2. SimSiam pretraining for 100 epochs: training loss (left) and kNN validation accuracy (right).

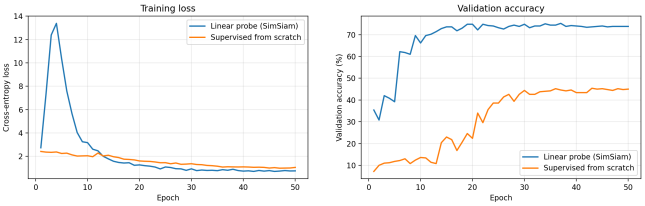


Figure 3. Linear probe on the frozen 100-epoch encoder vs. SFT on the 1,000 fold-0 images: training loss (left) and validation accuracy (right).

Method	Pretrain	Test acc. (%)	n
SFT from scratch	–	47.74±1.17	5 ft. seeds
SFT, tuned‡	–	55.66±0.93	5 ft. seeds
Linear probe	100 ep	75.80	1 run
Linear probe	200 ep	**80.98**±0.35	3 pt. seeds
Linear probe	1000 ep	**86.74**±0.23	1 run†

Table 1. Test accuracy with 1,000 labeled images. †One pretraining run, so ± covers finetuning seeds only. ‡Batch 64, 100 epochs, lr picked on validation from $\{0.03, 0.1, 0.3\}$.

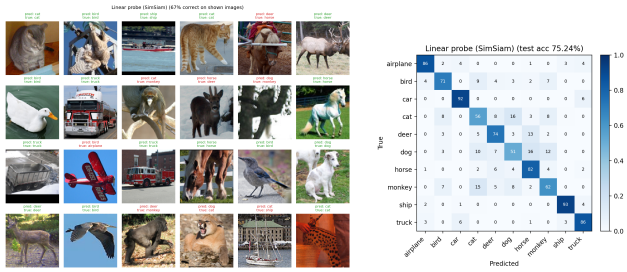


Figure 4. Linear probe on random test images (left; green: correct, red: wrong) and its confusion matrix on all 8,000 test images (right). Errors concentrate among animal classes.

5.1. SimSiam Pretraining

Figure 2 shows a 100-epoch pretraining run. The loss falls to about -0.78 during the 10-epoch warmup and ends at -0.848; the kNN accuracy rises steadily to 73.4%. The std of the normalized projector output ends at 0.02195, close to $1/\sqrt{2048} = 0.02210$, so the model has not collapsed.

Between epochs 50 and 80 the loss stays flat and even rises slightly ($-0.843 \rightarrow -0.837$), while the kNN accuracy climbs from 60.6% to about 70%. The loss only measures agreement between two views, not how useful the features are (see also Sec. 5.4).

5.2. Classification with and without Pretraining

Both classifiers use the same ResNet-18 and the same 1,000 labeled images (Fig. 3, Tab. 1). The linear probe passes 70% validation accuracy around epoch 10, while SFT stays near chance for 13 epochs and plateaus around 45%. After 200 epochs of pretraining, the linear probe beats SFT by 33.2 points, and still by **25.3 points** (95% CI 24.2–26.5, Welch p=10^{-8}) when SFT uses batch 64 (16 steps per epoch) and an lr tuned on validation.

Training longer does not close the gap either: at batch 512, SFT saturates at about 49–50% (200 epochs: training loss 0.24, so it overfits), and a 200-epoch linear probe adds only $+0.8$ points over 50 epochs. SimSiam therefore learns from the 100k unlabeled images what 1,000 labels alone cannot teach. Longer pretraining keeps helping with diminishing returns (75.8, 81.0 and 86.7% after 100, 200 and 1000 epochs), as in Tab. 4 of the original paper; in the 1000-epoch run loss and kNN accuracy plateau while the LR is above half its peak, and snapshot probes rise from 80 to 87% only as it anneals.

Setting	Loss	Std	kNN	Lin. probe
Baseline	-0.908	0.021	77.6	81.35
w/o stop-grad	-1.000	0	10.0	10.00
w/o predictor	-1.000	0	13.2	10.00

Table 2. Architecture ablations after 200 epochs (final-epoch values; one unseeded run each, so the baseline differs slightly from Tab. 3).

Error analysis. Errors are mostly between similar animals (cat↔dog, dog↔horse, deer↔horse; Fig. 4). Vehicles (car, ship, truck, airplane) reach 86–93% per-class accuracy.

5.3. Ablation: Stop-Gradient and Predictor

Both variants reproduce the collapse reported in the original paper (Fig. 5, Tab. 2). Within a few epochs the loss reaches its minimum of -1 and the output std drops toward 0, so every image is mapped to the same vector. After 200 epochs the linear probe scores *exactly* 10.00% with training loss $\ln 10 = 2.303$, which is chance level for 10 classes. The original paper's 0.1% is the same result for ImageNet's 1000 classes. After 100 epochs the probes still reach about 15%, because the backbone has not fully collapsed yet.

The two ablations produce nearly the same curves, as

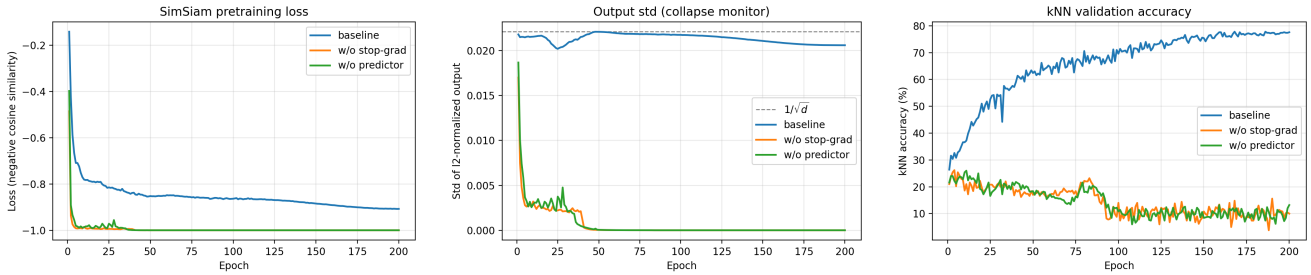


Figure 5. Ablating the stop-gradient and the predictor (200 epochs): training loss, std of the ℓ_2-normalized output (dashed: $1/\sqrt{d}$) and kNN accuracy. Both ablations collapse.

Setting	Loss	kNN	Lin. probe	BN probe
Baseline	−0.909	78.1	**80.98**±0.35	**81.40**±1.20
w/o color	−0.958	44.9	50.36±1.17	52.28±1.25
w/o crop	−0.995	19.3	17.50±2.19	24.77±1.54

Table 3. Augmentation ablation, 200 epochs, mean (±std) over 3 pretraining seeds. Loss and kNN are final-epoch values averaged over seeds.

the original paper's analysis predicts: without a predictor, the gradient of Eq. (1) has the direction of that of $\mathcal{D}(z_1, z_2)$, so removing the predictor in effect also removes the stop-gradient. The asymmetry from both components is required, in line with DirectPred [13]; collapse is deterministic, so one run per variant suffices.

5.4. Ablation: Augmentation

Lower loss, worse representation. Both variants reach a *lower* loss than the baseline, because the views are easier to align, yet their accuracy is far lower (Tab. 3). With the same probe and n=3 on both sides, removing color costs 30.6 points (p=1.8×10^{-4}), removing crop costs 63.5 points (p=3.0×10^{-4}), and no_crop is 32.9 points below no_color (p=1.6×10^{-4}). The standard probe underestimates no_crop (below), but the BN probe gives the same ordering (81.4, 52.3, 24.8; all pairs $p_{\text{Holm}} < 10^{-3}$). Cropping is therefore the more important augmentation. This agrees with SimCLR, but the drop is larger here: without negatives, a too-easy view pair leaves no other signal to learn from.

The standard probe fails on no_crop. With lr 30, the linear probe on no_crop's tiny-variance features never gets below training loss $\ln 10$, even with 200 epochs. The BN probe standardizes them (validation picks lr 1.0, vs. 0.03 for the baseline) and raises no_crop to 24.77% (+7.27, paired p=0.003), still 57 points below the baseline under the same probe; we use it for no_crop below.

no_crop gets worse with more training. no_color improves from 100 to 200 epochs (43.9 → 50.4), but no_crop moves the other way. With 3 seeded 100-epoch runs and the BN probe, its accuracy drops from 38.17 ± 2.39 to 24.77 ± 1.54 (Welch t=8.18, df 3.4, p=0.0023; 95% CI of the change $[-18.3, -8.5]$). The 100- and 200-epoch models come from separate cosine schedules, but the decline also shows *within* each 200-epoch run: no_crop's kNN accuracy peaks at 27–33% around epochs 15–21 and ends at 15–21%, while the baseline's keeps rising (Fig. 6, left).

Dimensional collapse, not complete collapse. The effective rank explains these results (Fig. 6, right). The 200-epoch no_stop-grad and no_predictor models are single points: their feature variance relative to the mean squared norm is 10^{-15} or smaller. no_crop is different. Its projector output z still spreads out (relative variance 0.93, nonzero std), but occupies only about 4–8 of 2048 dimensions, and its backbone erank is just 6.7 ± 2.8, versus 317.5 ± 2.4 for the baseline. This is *dimensional collapse* [10]. It worsens with training: for every seed, the backbone erank drops from 100 to 200 epochs (53.3 → 9.2, 120.3 → 7.4, 34.2 → 3.6) together with the accuracy. In terms of Eq. (2), a crop-free pair differs only in color and flip. Averaging over such augmentations gives an η_x that a few color-invariant directions can match, and the objective keeps shrinking the representation onto those directions. **Stop-gradient and the predictor prevent complete collapse, but not dimensional collapse.** In contrastive learning Jing *et al*. [10] found the opposite trigger, *strong* augmentation; here a *weak* one makes η_x so easy to match that a few directions suffice.

It is not overfitting: on 8,000 unseen images the loss is within 0.001 and the erank the same (9.1 vs. 9.0) as on training images. All t-tests above stay significant after Holm correction under accuracy, AUROC and F1 ($p_{\text{Holm}} \leq 0.035$).

5.5. What Does the Crop Contribute?

Table 4 reports the variants of Sec. 3, pretrained for 200 epochs with three seeds (shuffles: one). Measured as the fraction of the no_crop→baseline gap they recover under

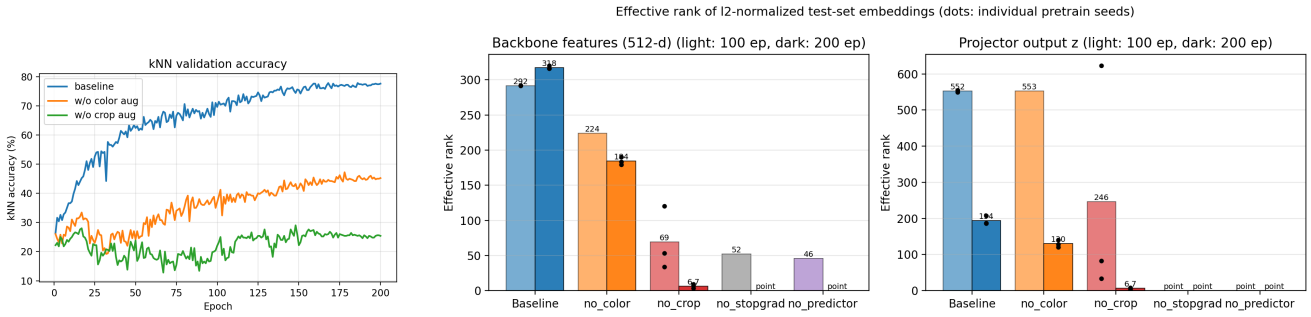


Figure 6. Left: kNN accuracy during augmentation-ablation pretraining (one representative 200-epoch run each). Right: effective rank of test-set embeddings for the backbone features and projector output z after 100 epochs (light) and 200 epochs (dark); bars are means and dots individual pretraining seeds. "point" means all images map to one point (relative variance $< 10^{-4}$), so no rank is defined.

Variant	n	erank	BN probe	AUROC
Crop (baseline)	3	318	**81.40**±1.20	97.80
mask	3	156	63.11±0.53	93.82
crop_in_place	3	75	48.66±1.68	88.31
zoom_out	3	74	36.58±2.09	81.88
shuffle	1	19	36.54	82.45
shuffle_shared	1	11	35.27	81.75
same_crop	3	5.5	27.65±5.33	77.81
No crop	3	6.7	24.78±1.54	72.33

Table 4. Replacing RandomResizedCrop (200 epochs). Mean (±std) over n pretraining seeds of the BN probe accuracy (each seed averaged over 5 finetuning seeds); erank is the backbone's; AUROC is macro one-vs-rest.

the BN probe, masking recovers 68%, crop_in_place 42%, zoom_out 21% and the two shuffles 19–21%; backbone erank broadly follows the same order. Every adjacent pair among baseline, mask, crop_in_place, zoom_out and no_crop differs significantly (Welch over pretraining seeds, $p_{\text{Holm}} \leq 0.014$; accuracy and F1).

The views must differ, not just the data. same_crop sees as many distinct crops as the baseline, yet it is as poor as no_crop (27.7 ± 5.3 vs. 24.8 ± 1.5, p=0.45; same loss and kNN accuracy), so data diversity and sample count do not explain the no_crop failure: the two views of one image must differ in *which content* they show.

Partial content and rescaling are complementary. Each component alone recovers only part of the gap: hiding content (crop_in_place) helps more than rescaling and shifting (zoom_out), and their sum (63%) falls well short of the full crop. A two-way OLS on the 12 models confirms the interaction: +20.9 points (95% CI 16.5–25.4, $p_{\text{Holm}} < 10^{-4}$; same for F1). In AUROC, which saturates near 98, the effects are additive (interaction −0.04). Scattered masking is the strongest single replacement (+14.5 over crop_in_place with the same visible area, p_{Holm}=0.014), probably because each view hides different parts of the whole object rather than one region, matching InfoMin [12]: views should share the object but differ in the rest.

A different layout is not a useful difference. shuffle and its control shuffle_shared end nearly level (36.5 vs. 35.3), so a different tile layout adds almost nothing over the seams themselves. Yet under a fixed reference encoder (the 1000-epoch baseline) shuffled views are *less* similar (cosine 0.82) than the baseline's crops (0.90): convolutions are translation-equivariant and global pooling discards position, so the network being trained can match permuted tiles from local statistics, whereas the crop demands invariance to scale and occlusion that local texture cannot provide. View similarity only flags the trivial case (0.98 for no_crop and same_crop); beyond it, the *kind* of invariance a pair demands matters, not how different the views look.

6. Limitations

(i) Erank is not a quality measure across settings: from 100 to 200 epochs the baseline's projector erank falls 552 → 194 and no_color's backbone erank 224 → 184 while accuracy rises. Our no_crop evidence rests on the backbone, where rank and accuracy fall together by large amounts. (ii) Nor is a falling std: the 1000-epoch baseline's std halves to 0.0100, yet its erank (392) and accuracy (86.7%) are our highest. (iii) With n=3 pretraining seeds the Welch df is only 2–4, though every effect is many standard deviations in size. (iv) The shuffles have one pretraining seed, and same_crop's seeds span 22–33 points; test-set McNemar tests (all $p_{\text{Holm}} < 10^{-4}$) ignore this pretraining variance and put same_crop below no_crop for one seed and above it for another. Mean-filled regions and tile seams may also act as shortcuts. (v) The tuned SFT's best lr (0.3) lies at the edge of our grid, and we did not fine-tune the SimSiam encoder

end to end.

7. Conclusion

With 1,000 labels, SimSiam features beat a tuned from-scratch network by 25 points; removing the stop-gradient or the predictor causes complete collapse. When the views are too similar (no cropping), SimSiam keeps a nonzero std and a lower loss, yet its features collapse onto a few dimensions and degrade with training; the effective rank and a properly scaled probe detect this. The views must differ in the content they show: a crop shared by both views is as harmful as no crop at all, and a difference the network can undo cheaply, such as rearranged tiles, does not help.

References

[1] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *International Conference on Machine Learning (ICML)*, pages 1597–1607, 2020. 1

[2] Xinlei Chen and Kaiming He. Exploring simple Siamese representation learning. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15750–15758, 2021. 1

[3] Adam Coates, Andrew Ng, and Honglak Lee. An analysis of single-layer networks in unsupervised feature learning. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 215–223, 2011. 1, 2

[4] Quentin Garrido, Randall Balestriero, Laurent Najman, and Yann LeCun. RankMe: Assessing the downstream performance of pretrained self-supervised representations by their rank. In *International Conference on Machine Learning (ICML)*, pages 10929–10974, 2023. 2

[5] Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre H. Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Daniel Guo, Mohammad Gheshlaghi Azar, Bilal Piot, Koray Kavukcuoglu, Rémi Munos, and Michal Valko. Bootstrap your own latent: A new approach to self-supervised learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. 1

[6] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016. 2

[7] Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross Girshick. Momentum contrast for unsupervised visual representation learning. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9729–9738, 2020. 2

[8] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16000–16009, 2022. 2

[9] Tianyu Hua, Wenxiao Wang, Zihui Xue, Sucheng Ren, Yue Wang, and Hang Zhao. On feature decorrelation in self-supervised learning. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 9598–9608, 2021. 2

[10] Li Jing, Pascal Vincent, Yann LeCun, and Yuandong Tian. Understanding dimensional collapse in contrastive self-supervised learning. In *International Conference on Learning Representations (ICLR)*, 2022. 2, 4

[11] Olivier Roy and Martin Vetterli. The effective rank: A measure of effective dimensionality. In *European Signal Processing Conference (EUSIPCO)*, pages 606–610, 2007. 2

[12] Yonglong Tian, Chen Sun, Ben Poole, Dilip Krishnan, Cordelia Schmid, and Phillip Isola. What makes for good views for contrastive learning? In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 6827–6839, 2020. 1, 5

[13] Yuandong Tian, Xinlei Chen, and Surya Ganguli. Understanding self-supervised learning dynamics without contrastive pairs. In *International Conference on Machine Learning (ICML)*, pages 10268–10278, 2021. 4

[14] Zhirong Wu, Yuanjun Xiong, Stella X. Yu, and Dahua Lin. Unsupervised feature learning via non-parametric instance discrimination. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3733–3742, 2018. 2